External linkage | nm a.out

Internal linkage | nm prog.o

- var D
  fun T

Storage class

⇒ Pointers - Basics (NULL/void) etc

Array - continuation

1.C

```
#----
main(){
    extern int var; //Pure Declaration, No defn
    pf("%d", var);
}
```

Error, Linking Error.

⇒ gcc -c 1.c ↵

⇒ nm 1.o ↵

main T
printf U
var U

⇒ gcc 1.o ↵

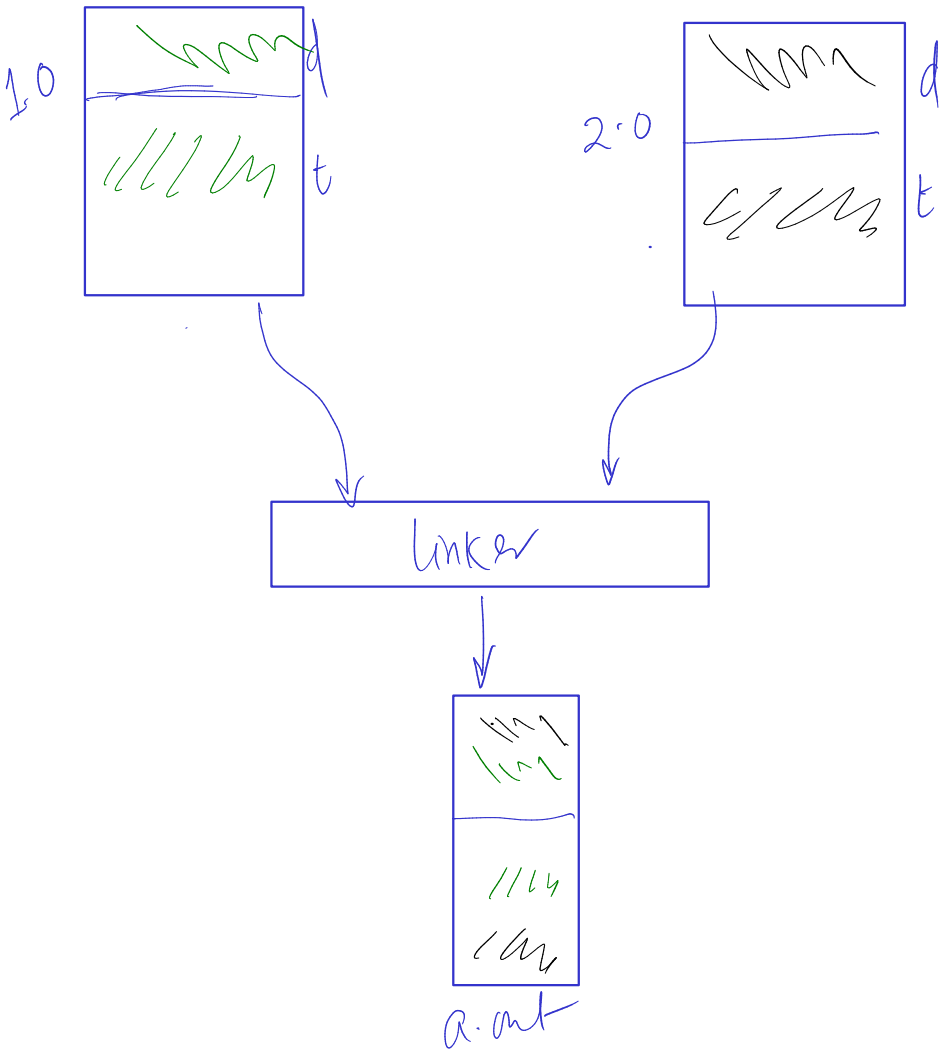
1.C

```
main(){
    extern int var;
//f1();
    printf("%d", var);
}
```

2.C

```
int var = 10;
f1()
{
=
}
```

$ gcc 1.c 2.c ↵



What's the practical use of extern ?

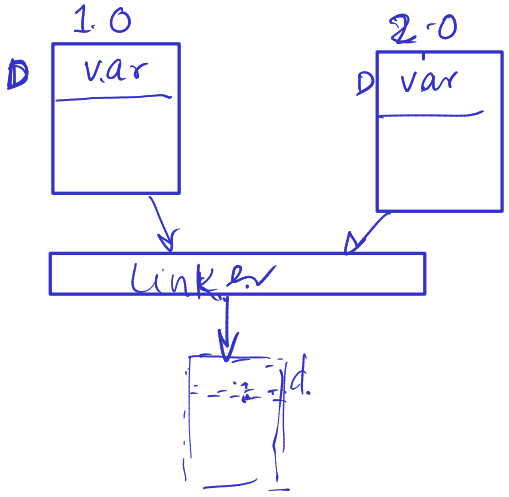
1.c

```
int var=10;
main() {
        f1();
        Pf("%d", var);
}
```

2.c

```
int var=20;
f1(){
        Pf("\n f1 .. var = %d", var);
}
```

$ gcc 1.c 2.c ↵



```
extern int V1; // Undefined
   int  V2; // Weak Symbol
int V3=0; // Strong Symbol
ham() {
      V1=V2=V3=30;
}
```

gcc -c 1.c

nm 1.o

V1 : U => Undefined Symbol

V2 : C => Common Symbol / Weak Symbol

V3 : D => Strong Symbol

```
main() { static int V=10;
         int V=20;
         Pf("%d", V);
}
```

Error

```
main() { extern int g;
         static int g;
         ≡
         Pf("%d", g);
}
int g=20;
```

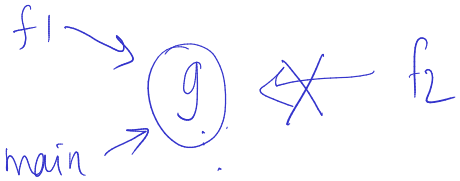
1.c

```
static int g=10;
main(){
    f1();
    f2();
    pf("%d", g);
}
f1(){
    ++g;
}
```

scope is limited visible within the file

2.c

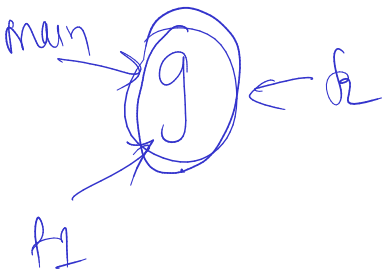
```
f2()
{
extern int g; // linking error
++g;
}
```



1.c

~~static~~ int g=10;

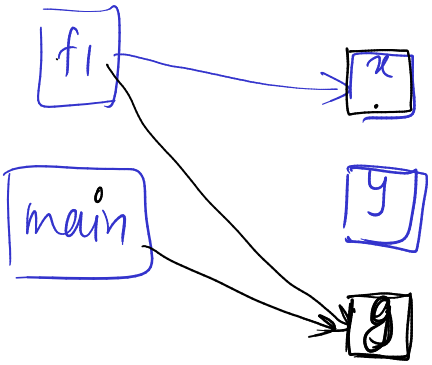
```
main(){ ++g;
}
f1(){ ++g;
}
```



2.c

```
f2()
{extern int g;
=
=
}
```

```
int g=100;
main() {
    f1();
    f1();
    f1();
}
f1() {
    static int x=10;
    auto int y=10;
    ++x; ++y;
    Pf("%d %d \n", x, y);
}
```



11 11
12 11
13 11

```
main() {
    int a=1;
    switch(a) { int b=5;
        case 1: printf("b=%d\n", b);
                break;
        default: Pf("In default\n");
    }
}
```

```
main() {
    int a=10;
    ++a;
    {
        int a=20;
        ++a;
        printf("a=%d\n", a); => 21
    }
    Pf("a=%d\n", a); // 11
}
```

```
for(int i=0; - ; i)
{
}
)
```

Block Structured